

Appunti 04/03/26 Bella copia

Il vecchio approccio: architettura stateful con sessioni

Il primo diagramma illustra il metodo tradizionale di gestione delle connessioni tra utenti e sistemi centralizzati. In questa configurazione, diversi client — identificati come Leonardo, Gabriela e Chiara — stabiliscono ciascuno un collegamento diretto verso un unico server centrale denominato Ferdinando. All'interno di questo server viene gestita la sessione, che funge da archivio centralizzato per le informazioni: il sistema mantiene una tabella delle sessioni in cui sono registrati i dati relativi a ogni singolo utente connesso, tra cui l'identità dell'utente, il suo username, i privilegi a lui assegnati e i claims. Tale meccanismo implica che l'autenticazione e la conservazione dello stato avvengano interamente sul server.

Memoria RAM e non scalabilità

Per ogni connessione attiva, il sistema crea e mantiene una sessione dedicata che risiede nella memoria RAM del server. All'interno di queste sessioni vengono archiviati dati specifici dell'utente, come lo username e i relativi claims. Poiché ogni utente — Leonardo, Chiara e Roberto — occupa uno spazio dedicato nella memoria RAM del server Ferdinando, il modello risulta intrinsecamente non scalabile.

Questa saturazione progressiva delle risorse hardware al crescere degli utenti collegati definisce l'inefficienza del sistema per carichi di lavoro elevati. Il termine bulk si riferisce proprio a questo ammasso o accumulo critico di dati che grava interamente sulle risorse del sistema: quando il numero di utenti cresce, il volume complessivo delle sessioni — contenenti username e claims — crea una massa di dati che può saturare la memoria disponibile, trasformando il server in un collo di bottiglia che impedisce al sistema di espandersi in modo efficiente per accogliere nuove connessioni. Per questo motivo, tale approccio viene adottato sempre meno nelle applicazioni web moderne.

Server stateful

Il server descritto è definibile come stateful. Un server stateful è un sistema che mantiene memoria delle interazioni precedenti con un determinato client, conservando i dati della sessione direttamente nelle proprie risorse locali. Come illustrato nel diagramma, il server Ferdinando associa a ogni utente uno stato specifico memorizzato nella propria RAM, dovendo ricordare chi sia l'utente e quali siano le sue autorizzazioni per tutta la durata del collegamento. Questo legame indissolubile tra il client e una specifica istanza del server rende il sistema non scalabile, poiché l'accumulo delle informazioni — il bulk — satura progressivamente la memoria disponibile all'aumentare delle connessioni.

Il nuovo approccio: JWT e architettura stateless

Il secondo gruppo di diagrammi illustra il nuovo approccio basato sull'utilizzo di token. Un token, nel contesto della sicurezza informatica e delle architetture moderne, è un oggetto digitale che funge da credenziale per identificare un utente senza che il server debba memorizzare la sua sessione in RAM. Il JWT — JSON Web Token — è lo standard utilizzato per lo scambio sicuro di informazioni tra un client e un server, permettendo di passare da un'architettura stateful a una stateless.

Il flusso prevede che il client invii le proprie credenziali (email e password) all'API del server. Quest'ultima, tramite il JWT Service, genera un token che viene restituito al browser. Il browser lo conserva nel proprio Local Storage e lo utilizza per le successive richieste, allegandolo ad ogni chiamata. Il server, ricevuta la richiesta con il token, lo valida tramite il JWT Service e recupera le informazioni dell'utente dal Repository, senza necessità di mantenere alcuno stato in memoria.

Token e JWT

Un token è un oggetto digitale che funge da credenziale per identificare un utente senza che il server debba memorizzare la sua sessione in RAM. Il JWT — JSON Web Token — è lo standard utilizzato per lo scambio sicuro di informazioni tra un client e un server, permettendo di transitare da un'architettura stateful a una stateless. Il JWT può essere descritto come un passaporto digitale temporale: non è cifrato, ma è firmato.

Struttura del JWT

Il JWT è composto da tre parti distinte: il preambolo, il body — detto anche payload — e la firma, chiamata signature. Queste tre parti vengono trasmesse come un'unica stringa e, all'occorrenza, possono essere separate mediante un'operazione di split che le restituisce come elementi di un array indicizzato.

Il ruolo della firma e del secret

All'interno del JWT Service risiede un elemento fondamentale noto come secret, una chiave privata conosciuta esclusivamente dal server. Il secret non coincide con il JWT stesso, ma viene utilizzato dal server per generare la firma, ovvero la terza parte del token. La firma ha la funzione di consentire al server di stabilire con certezza se il token che ha ricevuto sia stato generato da lui stesso o da un'entità esterna non autorizzata.

Flusso di validazione

Quando il client invia le proprie credenziali tramite un LoginDTO, queste raggiungono il JWT Service, che si occupa di generare e validare il token. Al momento di una richiesta successiva, il server riceve la stringa JWT, la suddivide nelle sue tre componenti ed estrae la parte con indice 2, ovvero la firma. Se il server riconosce la firma come propria — verificando la corrispondenza con il proprio secret — accetta il payload contenuto nella parte con indice 1, dove trova l'utente codificato insieme ai suoi claims e alle sue credenziali.

Appunti JWT e Concetti Correlati

Struttura di un JWT

Un JWT (JSON Web Token) è una stringa divisa in tre parti. La prima è il **preambolo (header)**, che contiene informazioni tecniche su come è stato creato il token, come l'algoritmo di firma usato — non è rilevante per la logica della nostra applicazione. La seconda è il **payload**, che contiene i dati dell'utente (i cosiddetti *claim*): è codificato in Base64, il che significa che chiunque può leggerlo, quindi non va mai inserito nulla di sensibile. La terza è la **firma**, che garantisce l'integrità del token e può essere verificata solo da chi possiede la chiave segreta del server.

Codificato vs Criptato

È importante non confondere questi due concetti. Un dato **codificato** è semplicemente trasformato in un formato diverso per facilitarne il trasporto — non c'è alcuna protezione, chiunque può decodificarlo senza bisogno di una chiave. Il payload JWT, ad esempio, è codificato in Base64. Un dato **criptato**, invece, è protetto da un algoritmo che richiede una chiave segreta per essere letto: senza di essa, il contenuto è incomprensibile. La crittografia tutela la riservatezza, la codifica no.

Il claim sub

Tra i claim standard del payload JWT, uno dei più importanti è **sub (subject)**, che identifica il soggetto del token, cioè l'utente loggato. Contiene tipicamente l'username, l'email o l'ID nel database. Grazie a questo campo, il server sa esattamente a chi appartiene il token senza dover fare ulteriori ricerche.

Gli Header HTTP

Gli header HTTP sono metadati che il client allega alla richiesta **prima del body**, sotto forma di coppie chiave-valore. Mentre il body contiene i dati veri e propri, gli header contengono il **contesto**: chi sta facendo la richiesta, in che formato sono i dati, che tipo di risposta si aspetta il client, e — nel caso del JWT — quale token di autenticazione sta presentando l'utente. Il server legge gli header prima ancora di aprire il body, per decidere come gestire la richiesta. Nel caso dell'autenticazione con JWT, il token va inserito nell'header `Authorization` con questo formato:

javascript

```
headers: {
```

```
  'Authorization': 'Bearer ' + token, // lo spazio dopo "Bearer" è obbligatorio
```

```
  'Accept': 'application/json',
```

```
  'Content-Type': 'application/json'}
```

Le Promise in JavaScript

Una **Promise** è un oggetto JavaScript che rappresenta il risultato futuro di un'operazione asincrona, cioè un'operazione che richiede tempo e non restituisce il risultato immediatamente. Nel contesto del JWT, ogni chiamata HTTP verso il server — come il login o una richiesta autenticata — è asincrona e restituisce una Promise. Il browser invia la richiesta e, senza bloccarsi, attende la risposta: quando arriva, se è andata bene esegue il `.then()`, se è fallita esegue il `.catch()`.

C'è però una cosa importante da ricordare: **`fetch()` considera la Promise soddisfatta anche quando il server risponde con un errore come 403 o 404**, perché dal punto di vista della comunicazione la promessa è stata mantenuta — la risposta è arrivata. Per questo motivo è necessario controllare manualmente `response.ok` o `response.status` per gestire correttamente i casi di errore applicativo.

Le Arrow Function

Le arrow function sono una sintassi alternativa e più compatta per scrivere funzioni in JavaScript, introdotta con ES6. Al posto della parola chiave `function`, si usa la freccia `=>`. Le regole di sintassi variano in base al numero di parametri e di istruzioni: con nessun parametro si usano le parentesi tonde vuote `()`, con un solo parametro le parentesi sono opzionali, con più parametri sono obbligatorie, e con più istruzioni si usano le parentesi graffe con il `return` esplicito.

Il metodo HTTP in fetch()

Per default, `fetch(url)` usa il metodo **GET**, perché è progettato per seguire lo standard HTTP e il caso d'uso più comune è la lettura di una risorsa. Va specificato esplicitamente solo quando l'operazione è diversa: **POST** per inviare nuovi dati (es. login, registrazione), **PUT** o **PATCH** per modificare dati esistenti, **DELETE** per cancellare una risorsa.

Generazione del JWT in Spring Boot

In Spring Boot, il token JWT si costruisce tramite il **pattern builder**: ogni metodo configura una parte del token e restituisce lo stesso oggetto `JwtBuilder`, permettendo di concatenare le chiamate in sequenza. Si parte con `.setClaims()` per il payload personalizzato, poi `.setSubject()` per l'utente, `.setIssuedAt()` e `.setExpiration()` per le date di emissione e scadenza, `.signWith()` per aggiungere la firma con la chiave segreta, e infine `.compact()` che costruisce e restituisce la stringa JWT finale.

L'Interceptor

Un interceptor è un componente che si inserisce nel flusso delle richieste HTTP prima che vengano elaborate. **Lato server in Spring Boot**, intercetta le richieste in entrata prima che raggiungano il controller, ed è tipicamente usato per verificare la validità del JWT. **Lato client in Angular**, funziona al contrario: intercetta le richieste in uscita prima che vengano inviate al server, aggiungendo automaticamente il token nell'header `Authorization`.

Il vantaggio principale è evidente: senza interceptor, ogni singola chiamata HTTP del progetto dovrebbe aggiungere manualmente il token — una duplicazione di codice enorme. Con l'interceptor, la logica si scrive **una volta sola** e si applica automaticamente a tutte le richieste.